

Atlas: A Self-Evolving Formal Foundation

Version 1 — The Graph-Theoretic Base

Atlas Cao

May 4, 2026

Abstract

We present a minimal formal foundation for a cognitive system in which the sole primitive is the act of distinction itself. Unlike traditional foundations (set theory, type theory, category theory) that posit multiple primitive concepts, our system begins from a single pre-formal event: the occurrence of a distinction. This event necessarily produces a primal graph—two nodes and an edge—which serves as the unique seed structure. On this graph, exactly four mutually irreducible operations are definable: focus, unfold, fold, and connect. These are proven complete: any non-trivial operation on legal graphs is expressible as a finite sequence of these four. Derivation is defined as graph transformation; contradiction is defined as a single syntactic pattern (a self-loop) and detected in $O(1)$. Equality is not assumed but explicitly registered as revisable equivalence declarations. Natural numbers arise from repeated folding over the primal graph. The foundation rigidly separates an immutable hard core (the logic of distinction) from nine explicitly marked optimization interfaces, enabling the system to evolve its own operational strategies without compromising logical integrity. We prove the non-collapse of the successor function and the completeness of the base operation set within their domain. This foundation is designed not merely to support reasoning but to enable the system that runs upon it to later restructure and optimize elements of the foundation itself.

Contents

1	The Starting Point	2
1.1	The Primal Acknowledgment	2
2	The Primal Graph	2
2.1	The Structure Produced by P	2
3	General Graphs	3
3.1	Definition of a Graph	3
3.2	Subgraphs and Equality	3
4	Labeled Nodes	4
5	The Four Base Operations	4
5.1	Focus α	4
5.2	Unfold ε	4
5.3	Fold σ	5
5.4	Connect λ	5
5.5	Completeness and Minimality	5

6	Derivation, Contradiction, and Equivalence	6
6.1	Derivation Steps	6
6.2	Contradiction: The Unique Termination Condition	6
6.3	Derivability	7
7	Equivalence Declarations	7
7.1	Definition and Registration	7
7.2	Revocation	8
8	Special Graphs and the Emergence of Number	8
8.1	The Two Boundary Graphs	8
8.2	Natural Numbers	8
8.3	Non-Collapse of Successor	8
8.4	Verification of Peano Axioms	9
9	The Hard Core and the Optimization Interfaces	9
9.1	The Immutable Hard Core	9
9.2	The Nine Optimization Interfaces	9
10	The Optimization Loop	10
10.1	Formalization of Self-Improvement	10
10.2	Self-Evolution of the Foundation	11
11	Relation to External Systems	11
11.1	Independence from Other Foundations	11
11.2	Embedding Other Formal Systems	11
11.3	Acknowledged Boundaries	11
12	Conclusion	12
A	Glossary of Symbols	12

1 The Starting Point

1.1 The Primal Acknowledgment

This foundation does not begin with an axiom. It begins with an **acknowledgment**.

Acknowledgment 1 (Primal Distinction P). *Distinction is already happening.*

You are reading this text. You distinguish “text” from “background.” You are thinking. You distinguish “this thought” from “other thoughts.” Any formal activity—including the attempt to deny this acknowledgment—already performs a distinction: it separates “what is to be denied” from the background against which the denial takes place.

Therefore, distinction is not something to be proved. It is the precondition under which proof becomes possible.

*We denote this irreducibly occurring act of distinction by P . P is not defined. P is **acknowledged**.*

2 The Primal Graph

2.1 The Structure Produced by P

Definition 2.1 (Primal Graph G_0). *When P occurs, it necessarily produces a single, indecomposable structure consisting of three interdependent components:*

- a distinguished item a ,
- another item b that is not a ,
- the relation r between them, which is the very act of their being distinguished.

This structure is called the **primal graph** and is denoted:

$$G_0 = \underset{a}{\bullet} \xrightarrow{r} \underset{b}{\bullet}$$

Theorem 2.1 (Indecomposability of G_0). G_0 cannot be decomposed into smaller independent existents.

Proof. Assume G_0 can be decomposed. Then the decomposition would contain either an isolated node without an edge, or an edge without endpoint nodes. An isolated node is impossible because a node is constituted precisely by being distinguished from another—which is exactly the edge. An edge without nodes is impossible because an edge is a relation, and a relation requires relata. Hence G_0 is indecomposable. $\square$

Corollary 2.2. Nodes and edges are not two independent primitive kinds. They are two inseparable faces of a single occurrence. There is no priority of one over the other.

Theorem 2.3 (Uniqueness of G_0). Under the acknowledgment of P , G_0 is the unique structure necessarily produced.

Proof. Any act of distinction simultaneously produces: (1) a focus directed upon something, (2) a background from which it is separated, and (3) the relation of separation itself. This yields exactly two nodes and one directed edge. Any other structure would fail to capture one of these three components. $\square$

3 General Graphs

3.1 Definition of a Graph

Definition 3.1 (Graph). A **graph** G is a triple (V, E, τ) where:

- V is a finite set of **nodes**,
- $E \subseteq V \times V$ is a set of directed **edges**,
- $\tau : E \rightarrow \mathcal{T}$ is a function assigning each edge a **type** from the registered type set $\mathcal{T}$.

Precondition: for every $(u, v) \in E$, $u, v \in V$.

Definition 3.2 (Type Set $\mathcal{T}$). The initial type set is:

$$\mathcal{T}_0 = \{ \text{points-to}, \text{associates}, \text{equivalent-to}, \text{constrains} \}$$

- **points-to**: directed reference from one node to another.
- **associates**: non-directional association between two nodes.
- **equivalent-to**: declares two nodes or subgraphs as equivalent.
- **constrains**: one node's state imposes a constraint on another.

3.2 Subgraphs and Equality

Definition 3.3 (Subgraph). $H \subset G$ if and only if $V_H \subseteq V_G$, $E_H \subseteq E_G$, and $\tau_H(e) = \tau_G(e)$ for all $e \in E_H$.

Definition 3.4 (Graph Equality). $G_1 = G_2$ if and only if there exists a bijection $f : V_1 \rightarrow V_2$ such that:

1. $\forall u, v \in V_1 : (u, v) \in E_1 \iff (f(u), f(v)) \in E_2$,
2. $\forall (u, v) \in E_1 : \tau_1(u, v) = \tau_2(f(u), f(v))$.

Graph equality is mechanically verifiable. In the worst case it reduces to the graph isomorphism problem; in practice, since every node carries a construction history (via labeled nodes; see Section 4), equality checking is typically constant-time via hashed construction identifiers.

4 Labeled Nodes

Definition 4.1 (Labeling). *Given a graph G and a subgraph $H \subset G$, a **labeling** operation produces a new graph G' in which H is replaced by a new node n_H , and a system-level edge (type **label**) connects n_H to H . We write:*

$$n_H \triangleleft H$$

and read it as “ n_H labels H ”.

Definition 4.2 (Labeled Node). *A node $n \in V_G$ is a **labeled node** if and only if there exists $H \subset G$ such that $n \triangleleft H$.*

A labeled node allows a complex subgraph to be treated as a single unit while preserving the ability to inspect its interior.

Definition 4.3 (Interface Nodes). *For a labeled node $n_H \triangleleft H$, the **interface nodes** $I_H \subseteq V_H$ are those nodes in H that, in the original graph G , had edges to or from nodes outside H . If unspecified, the default is:*

$$I_H = \{v \in V_H \mid \text{there exists an edge between } v \text{ and } G \setminus H \text{ in the original graph}\}.$$

5 The Four Base Operations

All operations take a graph as input and produce a graph as output. Each has a strict precondition. Exactly four mutually irreducible operations are definable.

5.1 Focus α

Definition 5.1 (Focus α).

$$\alpha(G, n) = G'$$

*where $n \in V_G$. G' is structurally identical to G , but node n is temporarily marked as the **current focus**. The focus mark does not affect graph equality.*

Precondition: $n \in V_G$.

Purpose: Designates the locus for subsequent operations. Without focus, no operation can be directed at a specific target.

5.2 Unfold ε

Definition 5.2 (Unfold ε).

$$\varepsilon(G, n) = G'$$

where $n \triangleleft H$. G' is constructed as follows:

1. Remove node n from G .
2. Insert all nodes and edges of H explicitly into G .
3. For every edge in the original G that was incident to n with an external node x :
 - if the edge was $x \rightarrow n$, create an edge from x to the corresponding interface node in I_H ;
 - if the edge was $n \rightarrow x$, create an edge from the corresponding interface node in I_H to x .

Precondition: n is a labeled node.

Purpose: Enter the interior of a packaged abstraction. Without unfold, labeled nodes remain permanently opaque.

5.3 Fold σ

Definition 5.3 (Fold σ).

$$\sigma(G, H) = G'$$

where $H \subset G$ and H satisfies the stability condition (Definition 5.4). G' is constructed as follows:

1. Determine the interface node set I_H of H .
2. Create a new node n_H .
3. Set $n_H \triangleleft H$.
4. Remove all internal nodes and edges of H from G , preserving only the external connection information.
5. Redirect all edges from $G \setminus H$ into H to point to n_H .
6. Redirect all edges from H to $G \setminus H$ to originate from n_H .

Precondition: H is stable (Definition 5.4).

Definition 5.4 (Stability). A subgraph $H \subset G$ is **stable** if and only if at least one of the following holds:

1. **Strongly Connected Component (SCC):** H is strongly connected (there exists a directed path from any node to any other), and no labeled node inside H has a pending unresolved internal structure that would alter H 's external connectivity upon unfolding.
2. **Unfolding Idempotence:** Two successive unfoldings of the labeled node $n_H \triangleleft H$ produce identical graph structures.

Purpose: Package a stabilized subgraph into a single unit. This is the formal act of abstraction—exiting details and forming a concept.

5.4 Connect λ

Definition 5.5 (Connect λ).

$$\lambda(G, n_1, n_2, t) = G'$$

where $n_1, n_2 \in V_G$ and $t \in \mathcal{T}$. G' adds a new edge (n_1, n_2) with type t to E_G .

Precondition: $n_1 \neq n_2$. An attempt to create a self-loop (n, n) is **rejected**: the derivation path is immediately marked as contradictory and terminated.

Purpose: Explicitly record a newly discovered relation between two existing nodes. Without connect, derivation can only rearrange—never discover new connections.

5.5 Completeness and Minimality

Theorem 5.1 (Completeness of $\{\alpha, \varepsilon, \sigma, \lambda\}$). Any non-trivial operation that produces a legal graph from G_0 can be expressed as a finite sequence of $\{\alpha, \varepsilon, \sigma, \lambda\}$.

Proof. By induction on the structural complexity $k = |V| + |E|$ of a legal graph.

Base case $k = 3$: The only legal graph is G_0 itself, obtained by zero operations.

Inductive step: Assume all legal graphs of complexity $\leq k$ are constructible via the four operations. Consider a legal graph G of complexity $k + 1$. G must arise from some graph G' of complexity $\leq k$ by one of the following:

- **Adding a node:** only σ produces a new node (by packaging a subgraph).
- **Adding an edge:** only λ produces a new edge.
- **Unfolding a labeled node:** only ε can do this.
- **Selecting a target:** α provides the focus needed for the above.

Each case corresponds to exactly one base operation. By the induction hypothesis, G' is reachable from G_0 via the four operations. Hence G is also reachable. $\square$

Theorem 5.2 (Minimality of $\{\alpha, \varepsilon, \sigma, \lambda\}$). *No proper subset of three operations is complete.*

Proof. We show each operation is irreplaceable.

- **Without α :** No target can be selected. Operations lack a specified locus.
- **Without ε :** Labeled nodes are permanently sealed. No internal structure can ever be accessed.
- **Without σ :** No abstraction can be formed. Graphs expand endlessly; conclusions cannot be packaged.
- **Without λ :** No new edges can be added. Derivation is restricted to permuting existing structure—genuine discovery is syntactically impossible.

Therefore, four operations are both necessary and sufficient. $\square$

6 Derivation, Contradiction, and Equivalence

6.1 Derivation Steps

Definition 6.1 (Derivation Step $\rightsquigarrow$). $G \rightsquigarrow G'$ if and only if either:

1. **Operation Application:** $G' = o(G)$ for some $o \in \{\alpha, \varepsilon, \sigma, \lambda\}$.
2. **Equivalence Application:** There exists a registered equivalence declaration $E = (P, Q, \text{meta}) \in \mathcal{E}$ such that G contains a subgraph matching pattern P (modulo hole nodes), and G' is the result of replacing that subgraph with the corresponding instance of Q .

Definition 6.2 (Derivation Sequence $\rightsquigarrow^*$). $\rightsquigarrow^*$ is the reflexive transitive closure of $\rightsquigarrow$:

$$G \rightsquigarrow^* H \iff \exists k \geq 0 : G \equiv G_0 \rightsquigarrow G_1 \rightsquigarrow \dots \rightsquigarrow G_k \equiv H.$$

6.2 Contradiction: The Unique Termination Condition

Definition 6.3 (Contradiction — Self-Loop). A graph G **contains a contradiction** if and only if there exists a node $n \in V_G$ such that $(n, n) \in E_G$ (a self-loop).

A self-loop encodes the statement “a node is distinguished from itself”—the graph-theoretic equivalent of $A \wedge \neg A$.

Theorem 6.1 (Contradiction Termination). *If a derivation path produces a graph G_k containing a self-loop, that derivation path is immediately terminated and produces no output.*

Proof. A self-loop is syntactically unstable and admits no further legal operation. The system refuses to continue. Detection is $O(|V|)$ in the number of nodes (constant time per node in adjacency-list representation). $\square$

Theorem 6.2 (Sources of Contradiction). *Contradiction can only arise from:*

1. An equivalence declaration $E \in \mathcal{E}$ whose application introduces a self-loop.
2. A conflict where two distinct derivation paths impose irreconcilable edge requirements that jointly force a self-loop.

Proof. The four base operations do not directly create self-loops: $\alpha, \varepsilon, \sigma$ do not add edges; λ is preconditioned to reject $n_1 = n_2$. Hence, a self-loop can only appear indirectly through an equivalence declaration that equates two subgraphs in a way that, under the current context, closes a loop on a node. $\square$

Corollary 6.3. *Every contradiction is auditable. The system can trace a self-loop back to the specific equivalence declaration or operation step that triggered it.*

6.3 Derivability

Definition 6.4 (Derivability $G \vdash_{\mathcal{E}} H$). *Under equivalence set $\mathcal{E}$, H is **derivable** from G , written $G \vdash_{\mathcal{E}} H$, if and only if there exists a derivation sequence $G \rightsquigarrow^* H$ such that:*

- *every equivalence application uses a declaration from $\mathcal{E}$,*
- *no graph in the sequence contains a self-loop.*

Theorem 6.4 (Derivation Fidelity). *If $G \vdash_{\mathcal{E}} H$ and G contains no self-loop, then H contains no self-loop.*

Proof. By induction on the length of the derivation sequence. Base case: zero steps, $H = G$, trivially true. Inductive step: assume the theorem holds for sequences of length $\leq k$. For a sequence of length $k + 1$, the last step is either an operation or an equivalence application. Operations do not introduce self-loops (Theorem 6.2). If an equivalence application would introduce a self-loop, the declaration is revoked before the step can be accepted (Theorem 7.1). Hence, no step introduces a self-loop into a previously self-loop-free graph. $\square$

This theorem is the formal basis of the **zero-hallucination guarantee**: from a consistent starting point, following only legal steps, one can only arrive at a consistent result—not as a probabilistic claim, but as a syntactic necessity.

7 Equivalence Declarations

7.1 Definition and Registration

Definition 7.1 (Equivalence Declaration E). *An **equivalence declaration** is a triple:*

$$E = (P, Q, \text{meta})$$

where:

- P and Q are **pattern graphs** (may contain hole nodes that match any subgraph),
- meta contains:
 - **equivalence type** (e.g., commutativity, associativity, idempotence, functional extensionality),
 - **source** (user-declared, Y -generated, or history-extracted),
 - **scope restriction**,
 - **registration timestamp**.

Definition 7.2 (Active Equivalence Set $\mathcal{E}$). $\mathcal{E}$ is the set of currently registered equivalence declarations.

Initial state: $\mathcal{E}_0 = \emptyset$.

The empty initial set means that even commutativity is not built-in. No two distinct structures are considered interchangeable until an explicit equivalence declaration is registered and verified.

Definition 7.3 (Registration). *A candidate equivalence declaration E is registered into $\mathcal{E}$ as follows:*

1. *The declaring agent (user, Y -engine, or W -engine) submits E .*
2. *The X -engine verifies E in a sandbox: it replays all historical derivation sequences while allowing applications of E , and checks whether any self-loop is introduced.*
3. *If no self-loop is introduced, E is added to $\mathcal{E}$ with a timestamp and source tag.*
4. *If a self-loop is introduced, E is rejected.*

7.2 Revocation

Theorem 7.1 (Equivalence Revocation). *If, during a derivation, the application of an equivalence declaration $E \in \mathcal{E}$ introduces a self-loop, then E is immediately removed from $\mathcal{E}$.*

Proof. The system backtracks from the contradiction to identify which equivalence declaration was applied to produce the self-loop. That declaration E is removed from $\mathcal{E}$. All derivation sequences that depended on E are marked as invalidated. $\square$

Equivalence declarations are thus **revisable contracts**, not eternal truths. If a declaration leads to a contradiction in an unforeseen context, it is retired.

8 Special Graphs and the Emergence of Number

8.1 The Two Boundary Graphs

Definition 8.1 (Zero Graph $\mathbf{0}$).

$$\mathbf{0} = (\emptyset, \emptyset, \emptyset)$$

$\mathbf{0}$ is the graph with no nodes and no edges. It represents the boundary state of “no distinction has been made”—an explicitly objectified emptiness.

Definition 8.2 (One Graph $\mathbf{1}$).

$$\mathbf{1} = (\{n\}, \{(n, n)\}, \tau(n, n) = \text{constrains})$$

$\mathbf{1}$ is the minimal contradiction graph: a single node with a self-loop. It serves as the terminal signal for a failed derivation.

8.2 Natural Numbers

Definition 8.3 (Successor S). *Let n be a labeled node representing a natural number. Its successor $S(n)$ is constructed as follows:*

1. $\varepsilon(n)$ — unfold the internal structure of n .
2. Apply α and λ to add a new distinction step on the unfolded structure.
3. σ — fold the result into a new labeled node.

Definition 8.4 (Natural Numbers $\mathbb{N}$). $\mathbb{N}$ is the smallest set containing $\mathbf{0}$ and closed under S . Thus:

$$0_{\mathbb{N}} \equiv \mathbf{0}, \quad 1_{\mathbb{N}} \equiv S(0_{\mathbb{N}}), \quad 2_{\mathbb{N}} \equiv S(1_{\mathbb{N}}), \quad \dots$$

8.3 Non-Collapse of Successor

Theorem 8.1 (Non-Collapse of Successor). *For all $n \in \mathbb{N}$, $S(n) \neq n$.*

Proof. Assume $S(n) = n$. Unfolding both sides: $\varepsilon(S(n))$ contains $\varepsilon(n)$ plus an additional distinction structure (at least one extra edge and node). Hence, the unfolded graphs have different complexities. By Definition 3.4 (graph equality), they cannot be equal. Contradiction. $\square$

This guarantees the natural numbers do not collapse into a finite cycle.

8.4 Verification of Peano Axioms

Theorem 8.2 (Peano Axioms). *The structure $(\mathbb{N}, 0_{\mathbb{N}}, S)$ satisfies the Peano axioms.*

Proof. 1. $0_{\mathbb{N}} \in \mathbb{N}$: by definition.
 2. $n \in \mathbb{N} \Rightarrow S(n) \in \mathbb{N}$: $\mathbb{N}$ is defined to be closed under S .
 3. $S(n) \neq 0_{\mathbb{N}}$ for all n : $S(n)$ is a labeled node; $\mathbf{0}$ is the empty graph. Structurally distinct.
 4. $S(n) = S(m) \Rightarrow n = m$: S is injective. Unfolding both successors and removing the outermost distinction layer yields identical structures for n and m .
 5. Induction: holds by the construction of $\mathbb{N}$ as the smallest set containing $\mathbf{0}$ and closed under finite application of S . □

Thus, arithmetic is not an appended layer but a necessary consequence of the structure of repeated distinction.

9 The Hard Core and the Optimization Interfaces

9.1 The Immutable Hard Core

The following components constitute the logical skeleton of the foundation. They cannot be altered by the system’s self-optimization without destroying the capacity for determinate reasoning.

Component	Content
P	Acknowledgment of the primal distinction
G_0	Primal graph: two nodes, one edge
$\{\alpha, \varepsilon, \sigma, \lambda\}$	The four base operations (kinds and signatures)
$\rightsquigarrow$	Derivation step definition
Self-loop	Contradiction detection
Equivalence format	Triple (P, Q, meta) structure
Graph equality	Structural bijection definition

9.2 The Nine Optimization Interfaces

The following are explicitly marked as replaceable during runtime. Each has a default implementation in Version 1 and can be replaced by a superior version discovered through operation history analysis by the M-engine and verified by the X-engine.

Interface 1: Focus Strategy — How to select the next focus node.

- *V1 Default*: Least-Recently-Operated Node.
- *Optimizable*: Task-dependent strategies (e.g., “node closest to target structure,” “node with highest degree,” “most unstable node”).

Interface 2: Unfolding Strategy — To what extent a labeled node is unfolded.

- *V1 Default*: Full Unfolding.
- *Optimizable*: Depth-limited unfolding, on-demand unfolding (only relevant parts), lazy unfolding (deferred until necessary).

Interface 3: Folding Trigger — When to judge a subgraph stable and trigger folding.

- *V1 Default*: SCC Stability (strongly connected component with no unresolved internal labels).

- *Optimizable*: Information-gain-based stability, goal-reachability-based stability, history-based stability (“unchanged for k steps”).

Interface 4: Connection Type Set — The set of allowed edge types $\mathcal{T}$.

- *V1 Default*: $\mathcal{T}_0 = \{\text{points-to, associates, equivalent-to, constrains}\}$.
- *Optimizable*: Y-engine proposes new types verified by X-engine for specific problem domains.

Interface 5: Matching Algorithm — How to find subgraphs matching a pattern P in a graph G .

- *V1 Default*: Exhaustive Scanning (node-by-node, edge-by-edge).
- *Optimizable*: Index-accelerated matching, hash-based subgraph lookup, learned heuristics for common patterns.

Interface 6: Replacement Strategy — How equivalence-based pattern replacement interfaces with the surrounding graph context.

- *V1 Default*: Strict Interface Matching (P and Q must have identical interface node counts and types).
- *Optimizable*: Flexible interface mapping when additional adapter equivalences are registered.

Interface 7: Modulation Strategy — When to refine (unfold more) or coarsen (fold more) a region.

- *V1 Default*: Fixed depth $d = 1$ per modulation step.
- *Optimizable*: Adaptive modulation based on derivation phase (exploration vs. convergence), graph size, and remaining computational budget.

Interface 8: Equivalence Discovery — How new equivalence declarations are generated.

- *V1 Default*: Y-engine searches in operator space for candidate equivalences.
- *Optimizable*: Automatic extraction from operation history (when two distinct operation sequences consistently produce identical results), cross-domain analogy mapping.

Interface 9: Fitness Weights — The weighting of dimensions in evaluating the fitness of a theorem or operational strategy.

- *V1 Default*: Uniform weights (all dimensions equal).
- *Optimizable*: Dynamic weighting by the S-engine based on current task distribution (e.g., prioritizing speed for time-critical tasks, prioritizing compression for memory-constrained tasks).

10 The Optimization Loop

10.1 Formalization of Self-Improvement

Definition 10.1 (Optimization Proposal $\mathcal{P}$). An *optimization proposal* is a quadruple:

$$\mathcal{P} = (\text{target}, \text{proposal}, \text{evidence}, \text{sandbox_result})$$

where:

- *target*: identifier of one of the nine optimization interfaces.
- *proposal*: formal definition of the new implementation.
- *evidence*: performance comparison data extracted from operation history.

- *sandbox_result*: outcome of isolated verification (see Rule 10.2).

Definition 10.2 (Sandbox Verification). *The X-engine verifies a proposal in an isolated environment by:*

1. *Temporarily replacing the targeted interface's default with the proposed implementation.*
2. *Running the modified system on a suite of historical tasks and synthetic test cases.*
3. *Checking that:*
 - *no self-loop is introduced in any derivation,*
 - *performance does not degrade in non-degenerate cases compared to the current version,*
 - *all results are reachable under the already registered equivalence declarations (no silent change in derivable conclusions).*

Definition 10.3 (Application and Rollback). *If sandbox verification passes:*

- *The interface default is updated to the proposal.*
- *The previous version is retained in a versioned history with timestamp.*

If sandbox verification fails, the proposal is rejected with a failure record.

*If a later degradation is detected (self-loop introduced or consistent performance loss), the system automatically rolls back to the previous version and marks the proposal as **verified-degraded**.*

10.2 Self-Evolution of the Foundation

Through the cycle of proposal generation (M-engine), sandbox verification (X-engine), and application/rollback, the default implementations of the nine interfaces can evolve without human intervention. The immutable hard core ensures the system remains a determinate reasoning engine. The replaceable interfaces allow it to become a continually improving one.

This is the formal realization of **adaptivity without nihilism**: the foundation is rigid where logic demands it, and fluid where optimization permits it.

11 Relation to External Systems

11.1 Independence from Other Foundations

This foundation does not presuppose ZFC set theory, category theory, type theory, or any other formal system. Its sole prerequisite is *P*, which cannot be denied without performing it. Hence, the foundation is self-grounding.

11.2 Embedding Other Formal Systems

Any formal system $\mathcal{T}$ (with axioms Γ and inference rules) can be embedded into this foundation as follows:

- Symbols of $\mathcal{T} \rightsquigarrow$ specific subgraph structures.
- Formulas of $\mathcal{T} \rightsquigarrow$ graphs.
- Axioms of $\mathcal{T} \rightsquigarrow$ equivalence declarations (registered with source tag $\mathcal{T}$ -axiom).
- Inference rules of $\mathcal{T} \rightsquigarrow$ admissible operation sequence patterns.
- Proofs in $\mathcal{T} \rightsquigarrow$ derivation sequences.

This allows the foundation to host and compare different formal theories within a single graph grammar.

11.3 Acknowledged Boundaries

The foundation does not claim omniscience. It is subject to:

- **Resource boundaries**: time and memory.

- **Gödelian incompleteness:** any system containing arithmetic has undecidable propositions.
- **Compilation uncertainty:** mapping from the physical world to graphs may admit multiple plausible schemas.

What the foundation guarantees is: when a derivation succeeds, it is syntactically faithful; when a contradiction arises, it is immediately detected and traced; when resources are exhausted, the system reports the boundary honestly without fabrication.

12 Conclusion

We have presented Version 1 of a mathematical foundation built upon a single pre-formal event—the occurrence of a distinction—and a single primal structure, the graph G_0 . Four irreducibly necessary operations, proven complete, allow the construction and transformation of any formalizable structure. Derivation, contradiction, and equivalence are defined with syntactic precision, yielding a deterministic, auditable reasoning engine. Natural numbers and arithmetic emerge necessarily from the repeated folding of the primal graph.

Crucially, the foundation is designed as a living system: an immutable hard core preserves the logic of distinction, while nine explicitly marked interfaces enable the system that runs upon this foundation to later restructure and optimize its own operational strategies. This completes the concept of a self-evolving formal base: one that is rigid enough to reason correctly, and fluid enough to improve itself indefinitely.

A Glossary of Symbols

Symbol	Meaning
P	Primal distinction — the acknowledged occurrence
G_0	Primal graph: two nodes, one edge
$G = (V, E, \tau)$	Graph: node set, edge set, type function
$\mathcal{T}$	Set of edge types
$\alpha(G, n)$	Focus: mark node n as focal point
$\varepsilon(G, n)$	Unfold: expand labeled node n
$\sigma(G, H)$	Fold: package subgraph H into a labeled node
$\lambda(G, n_1, n_2, t)$	Connect: add edge of type t between n_1, n_2
$G \rightsquigarrow G'$	Single derivation step
$G \rightsquigarrow^* H$	Derivation sequence
$E = (P, Q, \text{meta})$	Equivalence declaration
$\mathcal{E}$	Active set of equivalence declarations
$\mathbf{0}$	Zero graph: $V = \emptyset, E = \emptyset$
$\mathbf{1}$	One graph: minimal contradiction (self-loop)
$\mathbb{N}$	Natural numbers
$S(n)$	Successor function
$\mathcal{P}$	Optimization proposal (quadruple)